

Fundamentals of Computers

How a computer works — from a switch to a sentence

BSc Computer Science (First Year) · Detailed notes with diagrams · 2-mark & 14-mark questions

Learning objectives. After studying these notes you should be able to:

- Define a computer and state what a transistor is.
- Read and write binary, and explain why computers use it.
- Explain how text, numbers and images are represented as bits.
- Describe logic gates and how they perform arithmetic.
- Explain the CPU's fetch–decode–execute cycle and the levels of abstraction.

1. What is a computer?

Definition. A **computer** is an electronic device that accepts **data** and instructions (input), processes that data according to a stored set of instructions called a **program**, produces useful **information** (output), and can **store** both data and results.

At the deepest level a computer does only one thing: it decides between two states — **ON** and **OFF** — billions of times per second. Every program, image, message and calculation is built up from that single, repeated choice. The path from that tiny decision up to something meaningful is the story of this lecture, told as a stack of layers.

The basic cycle every computer follows is often written as **IPO + Storage**: **Input** → **Processing** → **Output**, with a Memory/Storage unit supporting every step.

2. The switch: the transistor

The hardware of a computer is built from one component repeated at enormous scale: the **transistor**. A transistor is an electronic **switch with no moving parts**. A small voltage applied to its **gate** decides whether current is allowed to flow through it.

- **Gate ON** → current flows → the transistor holds a **1**.
- **Gate OFF** → no current → it holds a **0**.

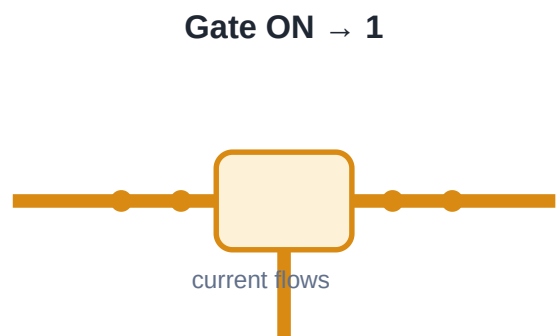
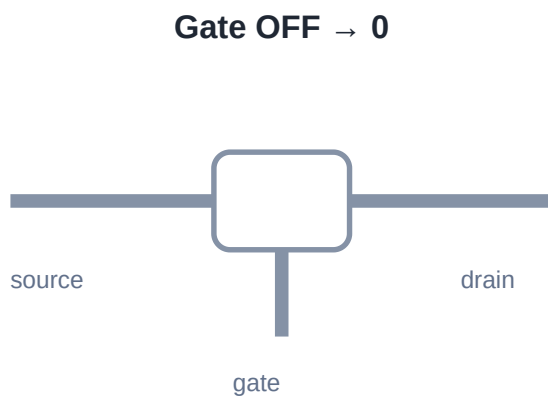


Fig 1. A transistor acts as a controllable switch: OFF stores 0, ON stores 1.

An **Integrated Circuit (IC)**, or 'chip', is simply millions to tens of billions of these transistors manufactured onto a single small piece of silicon. This is what people mean by 'what is inside a computer chip'.

3. The binary system: 1s and 0s

Because a switch has exactly two positions, a computer naturally speaks a two-symbol language called **binary**. A **1** means 'high voltage / on'; a **0** means 'low voltage / off'.

- A single **bit** (binary digit) is one switch — it can store only 2 values.
- Eight bits together form one **byte**, which can store $2^8 = 256$ patterns (0 to 255).
- Each position has a **place value** that doubles from right to left: 1, 2, 4, 8, 16, 32, 64, 128.

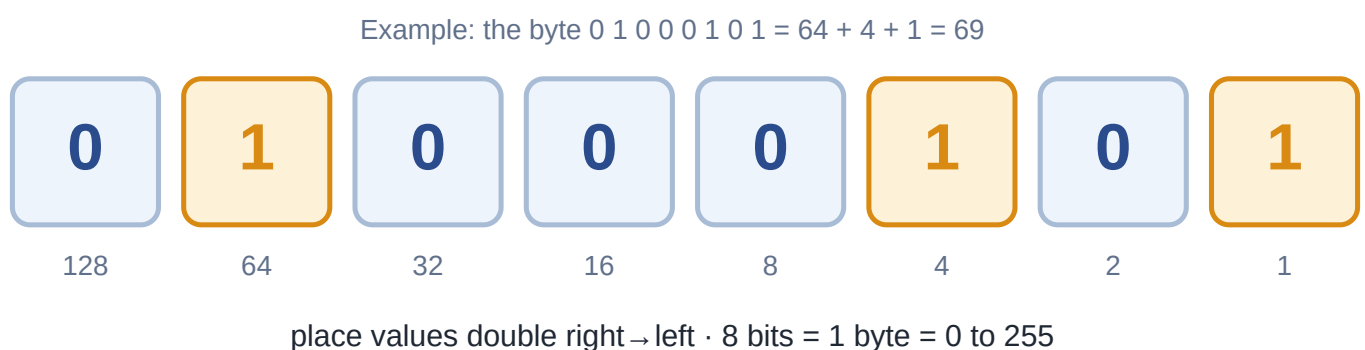


Fig 2. Reading a byte: switched-on place values are added together to give the decimal number.

Why binary? Electronic components are reliable at telling apart just two clear states (high/low voltage). Two states are cheap to build, resist noise, and map perfectly onto true/false logic — so binary is the most dependable choice.

4. Data representation: bits become meaning

A number is a pattern of bits. To store anything else, we simply **agree** on a code that maps patterns to meanings. For text, the standard **ASCII** code assigns each character a number: 'A' = 65, 'B' = 66, and so on.



letter → number → 8-bit binary (ASCII code)

Fig 3. Encoding text: a character maps to a number, which is stored as binary (ASCII).

The same idea represents everything else: an image is numbers for the colour of each pixel; sound is numbers for air pressure taken many times a second; a video is many images per second. **All data is ultimately bytes plus an agreed meaning.**

5. Logic gates: making decisions

Storing bits is not enough — a computer must **reason** with them. Small groups of transistors are wired into **logic gates**: circuits that take input bits and produce an output bit by a fixed rule.

Gate	Rule	Truth (A,B → out)
AND	1 only if BOTH inputs are 1	00 → 0, 01 → 0, 10 → 0, 11 → 1
OR	1 if EITHER input is 1	00 → 0, 01 → 1, 10 → 1, 11 → 1
NOT	flips the single input	0 → 1, 1 → 0
XOR	1 only if inputs DIFFER	00 → 0, 01 → 1, 10 → 1, 11 → 0
NAND	opposite of AND (universal)	00 → 1, 01 → 1, 10 → 1, 11 → 0

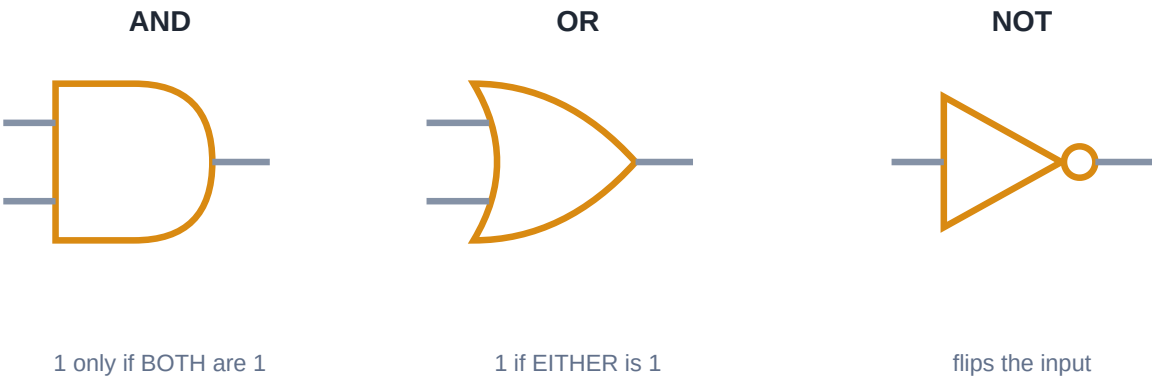
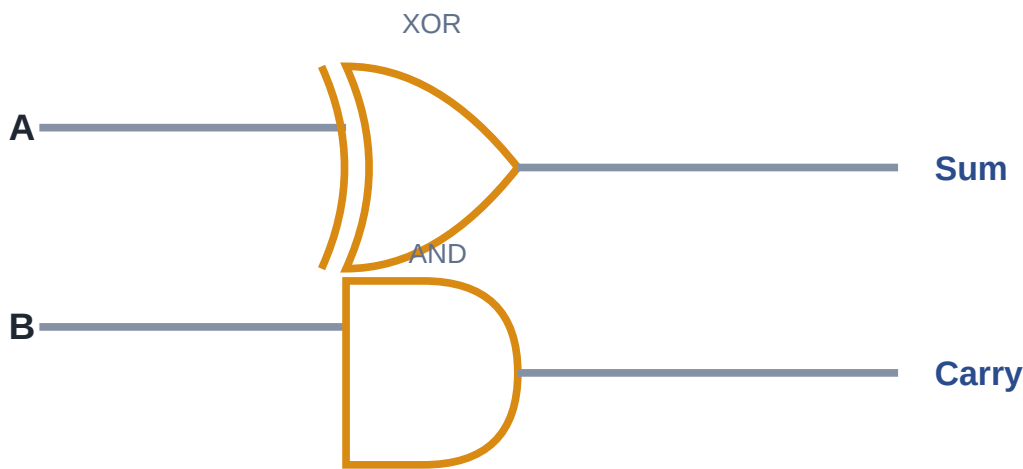


Fig 4. The standard symbols for the AND, OR and NOT gates.

6. From logic to arithmetic

Gates can be combined to calculate. Feeding two bits into an **XOR** gate and an **AND** gate builds a **half adder**: the XOR gives the **sum** bit and the AND gives the **carry**. In binary $1 + 1 = 10$ — carry 1, sum 0. Chaining many such adders lets a computer add any numbers, which is the seed of all computation.



Half adder: $1 + 1 = 10$ (carry 1, sum 0)

Fig 5. A half adder built from one XOR and one AND gate.

7. The CPU and the fetch–decode–execute cycle

Millions of gates are organised into the **Central Processing Unit (CPU)** — the 'brain' of the computer. The CPU runs one endless cycle:

- **Fetch** — read the next instruction from memory.
- **Decode** — the Control Unit works out what the instruction means.
- **Execute** — the instruction is carried out (e.g. the ALU adds two numbers).

A **program** is simply a list of such instructions stored in memory. This design — instructions and data held together in memory — is called the **stored-program (Von Neumann) concept**.

the CPU repeats this cycle billions of times per second



Fig 6. The fetch–decode–execute cycle repeats continuously while the computer runs.

8. Levels of abstraction: how a computer 'understands' us

A computer does not understand human language; instead, layers of **translation** convert what we write, step by step, down to voltage. Each layer hides the complexity of the one below it.

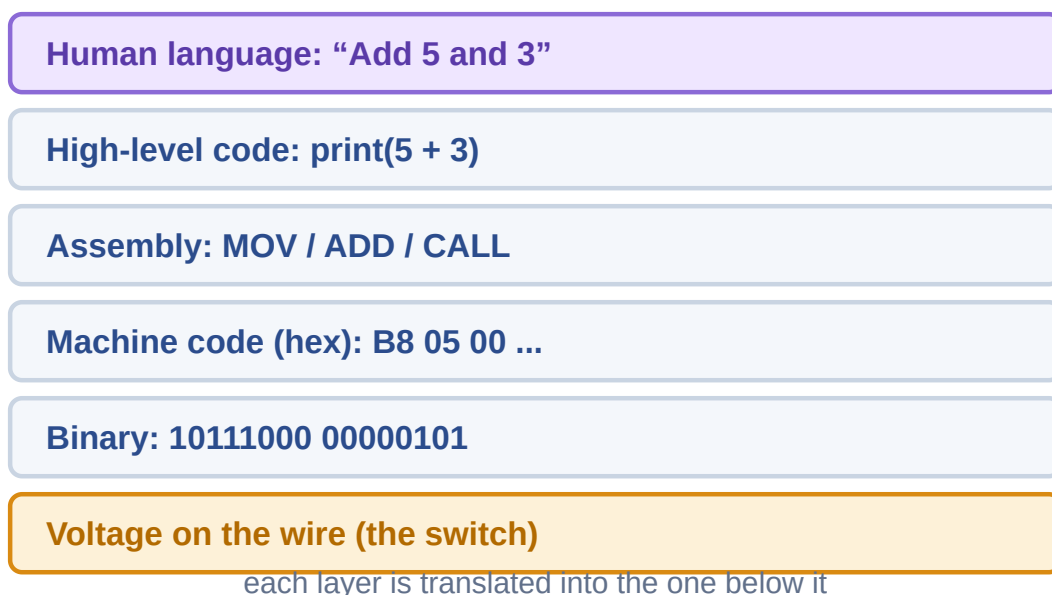


Fig 7. The abstraction stack: human meaning is translated down to switch states.

Chapter summary. switch (transistor) → bit → byte → encoded meaning → logic gate → arithmetic → CPU cycle → layers of translation. That is a computer, bottom to top.

2-Mark Questions (Short Answer)

Answer in 2–4 lines. These test definitions and single facts.

Q1. Define a computer.**2 marks**

A computer is an electronic device that accepts data and instructions as input, processes the data using a stored program, produces information as output, and can store data and results.

Q2. What is a transistor?**2 marks**

A transistor is an electronic switch with no moving parts. A small voltage on its gate controls whether current flows through it, letting it store a 0 (off) or 1 (on).

Q3. What is a bit and a byte?**2 marks**

A **bit** is the smallest unit of data — a single binary digit, 0 or 1. A **byte** is a group of 8 bits and can represent 256 (2^8) values, 0 to 255.

Q4. Why do computers use the binary number system?**2 marks**

Because electronic switches reliably distinguish only two states (high/low voltage). Two states are cheap to build, resist noise, and map directly onto true/false logic.

Q5. What is ASCII?**2 marks**

ASCII (American Standard Code for Information Interchange) is a standard code that assigns a number to each character, e.g. 'A' = 65, so text can be stored as binary.

Q6. Name the three basic logic gates and state each rule.**2 marks**

AND — output 1 only if both inputs are 1; OR — output 1 if either input is 1; NOT — flips its single input ($0 \rightarrow 1, 1 \rightarrow 0$).

Q7. What is an Integrated Circuit (IC)?**2 marks**

An IC (chip) is a small piece of silicon containing millions to billions of transistors and their connections, manufactured together as one unit.

Q8. What is a program?**2 marks**

A program is a list of instructions, stored in memory, that tells the computer exactly what steps to perform to complete a task.

Q9. State the three steps of the machine cycle.**2 marks**

Fetch (get the next instruction from memory), Decode (work out its meaning), and Execute (carry it out). The cycle then repeats.

Q10. What does the ALU do?

2 marks

The Arithmetic Logic Unit performs all arithmetic operations (add, subtract, etc.) and logic operations (compare, AND, OR) inside the CPU.

14-Mark Questions (Long Answer)

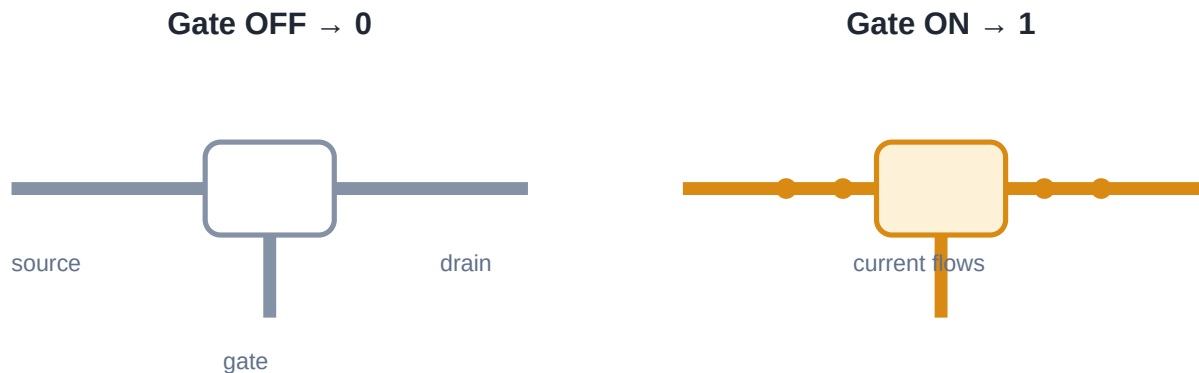
Long answers should include a definition, well-organised points, at least one labelled diagram, and a short conclusion.

Q1. Explain how a computer represents and processes information, starting from a single transistor up to a meaningful result. Support your answer with diagrams.

14 marks

Introduction. A computer stores and processes all information using just two states — on and off — represented as 1 and 0. Complexity is built by layering this simple idea.

1. The transistor (the switch). The transistor is the building block: an electronic switch whose gate voltage decides whether current flows. ON stores 1, OFF stores 0. Billions of transistors on one chip form an Integrated Circuit.



Transistor as a switch: OFF = 0, ON = 1.

2. Binary and the byte. One switch is one bit. Eight bits form a byte, storing 256 patterns. Place values double from right to left, so any number 0–255 can be built by switching bits on.

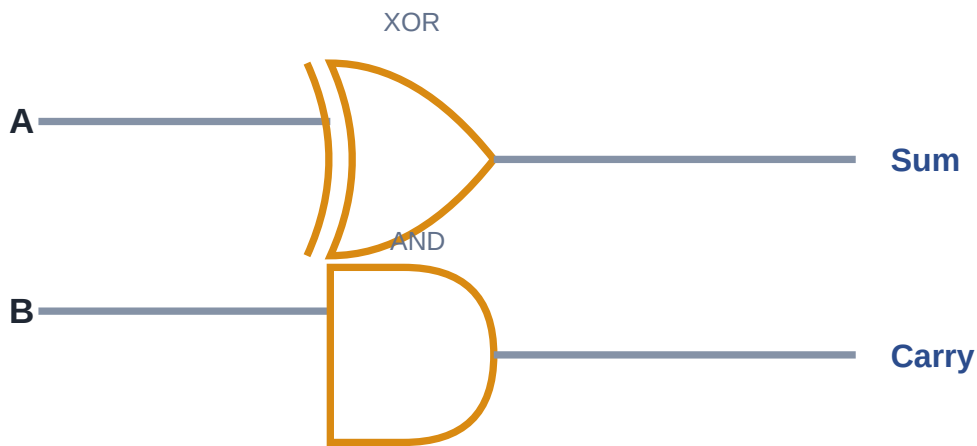
Example: the byte 0 1 0 0 0 1 0 1 = $64 + 4 + 1 = 69$



place values double right → left · 8 bits = 1 byte = 0 to 255

A byte and its place values.

3. Data representation. Numbers are patterns of bits; by agreeing on codes such as ASCII, the same patterns represent letters, and by extension images (pixel colours) and sound (pressure samples). **4. Logic gates and arithmetic.** Transistors are wired into logic gates (AND, OR, NOT) that make decisions, and combinations such as the half adder perform arithmetic.



Half adder: $1 + 1 = 10$ (carry 1, sum 0)

A half adder performs binary addition.

5. Processing. The CPU fetches, decodes and executes instructions in an endless cycle, using the ALU for calculations, turning stored bits into results.

Conclusion. From a single switch, through binary, encoding, logic and the CPU cycle, a computer converts simple on/off states into meaningful information.

Q2. What are logic gates? Describe the AND, OR, NOT, XOR and NAND gates with their truth tables, and explain how gates can be used to perform addition.

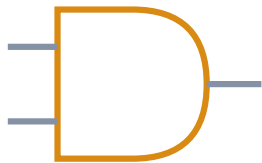
14 marks

Definition. A logic gate is a small circuit built from transistors that takes one or more input bits and produces an output bit according to a fixed rule. Gates are the basic decision-making elements of digital hardware.

The gates and their rules.

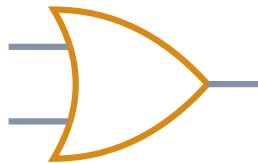
Gate	Rule	Truth table (inputs → output)
AND	1 only if both inputs are 1	00 → 0, 01 → 0, 10 → 0, 11 → 1
OR	1 if at least one input is 1	00 → 0, 01 → 1, 10 → 1, 11 → 1
NOT	inverts the single input	0 → 1, 1 → 0
XOR	1 only if the inputs differ	00 → 0, 01 → 1, 10 → 1, 11 → 0
NAND	NOT-AND; opposite of AND	00 → 1, 01 → 1, 10 → 1, 11 → 0

AND



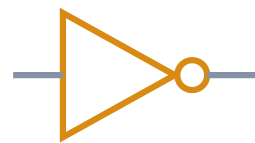
1 only if BOTH are 1

OR



1 if EITHER is 1

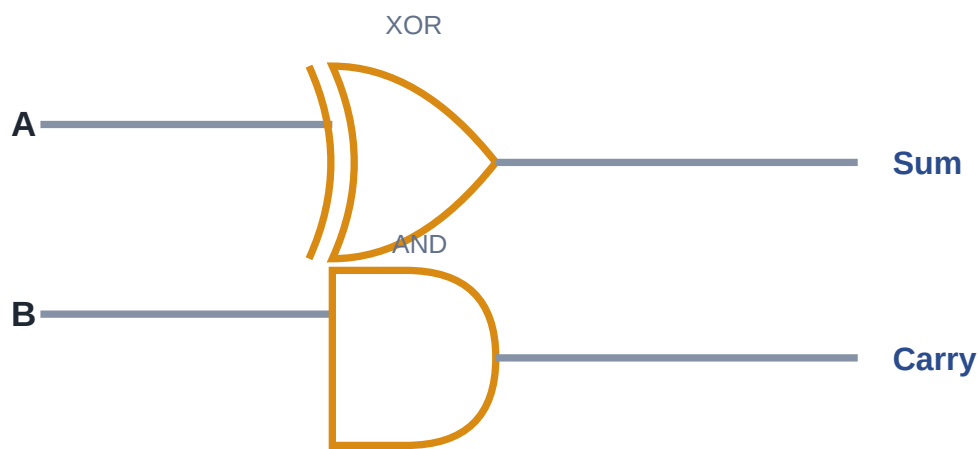
NOT



flips the input

Symbols for the AND, OR and NOT gates.

Using gates to add. An XOR gate and an AND gate together form a **half adder**. The XOR output gives the sum bit and the AND output gives the carry bit. For example, 1 + 1 gives sum 0 and carry 1, i.e. binary 10.



Half adder: $1 + 1 = 10$ (carry 1, sum 0)

Half adder: XOR gives the sum, AND gives the carry.

Conclusion. A few simple, rule-based gates, combined cleverly, can carry out arithmetic — which is why gates are called the verbs of the machine.

Q3. Describe the CPU and explain the fetch–decode–execute cycle. What is meant by the stored-program (Von Neumann) concept?

14 marks

The CPU. The Central Processing Unit is the brain of the computer, built from millions of logic gates. Its main parts are the **Control Unit** (directs operations), the **Arithmetic Logic Unit** (performs calculations and comparisons), and **registers** (tiny, very fast storage for the values being worked on).

The machine cycle. The CPU runs one instruction after another by repeating three steps:

- **Fetch** — the next instruction is copied from memory into the CPU.
- **Decode** — the Control Unit interprets what the instruction means.
- **Execute** — the instruction is carried out, e.g. the ALU adds two numbers and the result is stored.

This cycle repeats billions of times each second.

the CPU repeats this cycle billions of times per second



The fetch–decode–execute cycle.

Stored-program (Von Neumann) concept. In this design, both the program's instructions and its data are stored together in the same memory. The CPU fetches instructions from memory just like data. This idea — proposed in the 1940s — is the basis of nearly all modern computers, because it lets a machine run any program simply by loading different instructions into memory.

Conclusion. The CPU's simple, relentless fetch–decode–execute loop, combined with the stored-program idea, is what makes a computer a flexible, general-purpose machine.

Q4. Explain, with a diagram, how a program written in a human-readable language is eventually carried out as electrical signals. Why do we say a computer works through 'levels of abstraction'?

14 marks

Introduction. A computer does not understand human language. Instead, what we write is translated, step by step, into a form the hardware can act on. Each stage is a 'level of abstraction' that hides the details of the level below it.

The layers, top to bottom.

- **Human intention / language** — what we want, e.g. 'add 5 and 3'.
- **High-level language** — code such as `print(5 + 3)`, easy for people to read.
- **Assembly language** — short mnemonic instructions closer to the machine.
- **Machine code** — the binary instructions the CPU actually executes.
- **Binary / voltage** — the 1s and 0s, which are simply high and low voltages on wires (the transistor switches).

Human language: "Add 5 and 3"

High-level code: `print(5 + 3)`

Assembly: MOV / ADD / CALL

Machine code (hex): B8 05 00 ...

Binary: 10111000 00000101

Voltage on the wire (the switch)

each layer is translated into the one below it

Translation from human meaning down to voltage.

Why abstraction matters. Abstraction lets a programmer work in an easy, human language without knowing the electrical details, while translators (compilers and assemblers) handle the conversion. It makes large systems manageable: each layer can be understood, built and fixed on its own.

Conclusion. 'Understanding' is really a controlled fall from meaning to metal — every layer translating into the next until the request becomes voltage on a wire.